COMP/SOFT 4902/4912
Graduation Design Project

FEYZİYE SCHOOLS FOUNDATION
IŞIK UNIVERSITY

Tumor Segmentation Using Deep Learning

Dilara Sönmez / 20COMP1015, Abdelrahman Elmughrabi / 21SOFT1070

SYSTEM DESIGN DOCUMENT

SYSTEM DESIGN DOCUMENT

1. Introduction

This System Design Document presents the high-level software architecture of the proposed system, Tumor Segmentation Using Deep Learning. The system is being developed as a desktop-based decision support application for breast image analysis. Its main purpose is to process breast imaging data, localize suspicious regions, segment possible tumor areas, classify the detected finding as benign or malignant, and present the result together with a visual explanation of the model's decision. This design is consistent with the scope defined in the project proposal, where the project was described as a desktop application combining segmentation, classification, and explainability in a single workflow.

The architectural design was shaped by the main non-functional goals of the project. In particular, the system was designed with reliability, interpretability, usability, modularity, and maintainability in mind. These qualities are especially important because the project operates on medical imaging data, where outputs should not only be accurate but also understandable and easy to inspect. The proposal also emphasized robustness against image variability and the need to make predictions more transparent through explainability techniques.

1.1 Purpose of the System

The purpose of this system is to support the analysis of breast imaging data by offering an AI-assisted workflow for suspicious region detection, tumor segmentation, and benign/malignant classification. In simple terms, the system is intended to help the user upload a breast image, run it through a trained deep learning pipeline, and receive an interpretable result that highlights the suspicious area and explains why the model produced

its prediction. This is aligned with the project proposal, which defines the final product as a functional desktop application for breast image analysis with explainability output.

Another important purpose of the system is to bring the different stages of analysis into one consistent application instead of treating them as disconnected tasks. Rather than using one model for localization and another completely separate environment for classification, the project aims to connect these steps in a single pipeline. This makes the system more practical for demonstration and easier to evaluate as one integrated solution. The proposal specifically states that the project's unique aspect is combining segmentation and classification while also adding explainability to avoid a black-box style output.

1.2 Design Goals

1.2.1 Usability

Usability is one of the main design goals of the system. The application is intended for medical professionals who may not have a technical background in artificial intelligence or software tools. For this reason, the system is designed with a simple and straightforward desktop interface that allows the user to complete the full workflow, from image loading to result interpretation, without requiring prior training. The interaction flow is kept as clear as possible, and the interface is structured around the main tasks of the user rather than the internal complexity of the underlying models. In addition, the system is designed to provide error messages that are understandable and actionable, so that users can respond to problems without needing technical knowledge.

1.2.2 Reliability

Reliability is another major design goal because the system processes medical image data and is expected to behave consistently. The software is designed to handle invalid inputs, unsupported file types, and model inference failures gracefully without crashing. Instead of terminating unexpectedly, the system should detect such cases and inform the user in a controlled manner. In addition, the inference pipeline is designed to be reproducible, meaning that the same image input should always produce the same segmentation and classification output under the same system conditions. This consistency is important for both user trust and evaluation of the system.

1.2.3 Performance

The system is designed to provide results within a reasonable time on a standard personal computer. Since the project is intended as a practical desktop application, the runtime workflow should remain efficient enough for demonstration and regular use. The full pipeline, including preprocessing, segmentation, classification, and explainability generation, should therefore be completed without excessive delay. The design does not depend on GPU availability in order to run, which makes the application more accessible in ordinary computing environments. However, the architecture also allows the use of GPU acceleration when available to improve processing speed.

1.2.4 Supportability

Supportability is addressed through a modular system structure. The software is divided into clearly separated components such as the user interface, preprocessing module, model

inference module, and explainability module. This separation makes the system easier to test, debug, and update over time. It also allows individual components to be improved or replaced without requiring major changes in the rest of the application. As a result, the system becomes easier to maintain during development and more flexible for future enhancements.

1.2.5 Implementation

The implementation goal of the project is to build the system using widely accepted and well-supported technologies that are appropriate for machine learning-based desktop software. The system is implemented in Python, which provides a practical environment for integrating image processing, deep learning, and interface development. The desktop interface is developed using a standard Python GUI framework, while the deep learning components are implemented using PyTorch. This choice supports both rapid development and easier integration between the software components.

1.2.6 Legal and Ethical Compliance

Legal and ethical compliance is an important design goal because the system uses medical imaging data. All datasets selected for training and evaluation must be publicly available and used according to their respective licenses. The system is also designed so that no patient-identifiable information is stored, exposed, or processed during application use. This helps ensure that the project remains aligned with responsible data handling practices and avoids unnecessary privacy risks.

1.3 Definitions, Acronyms, and Abbreviations

AI: Artificial Intelligence

CNN: Convolutional Neural Network

DICOM: Digital Imaging and Communications in Medicine, a standard format for medical images

Grad-CAM: Gradient-weighted Class Activation Mapping, an explainability method that highlights image regions affecting a model's decision

GUI: Graphical User Interface

IoU: Intersection over Union, a metric used to evaluate segmentation overlap

Mask: A pixel-level output showing the region predicted as tumor or suspicious tissue

Preprocessing: The set of image preparation steps applied before model inference, such as resizing, normalization, and contrast adjustment

RAD: Requirements Analysis Document

ROI: Region of Interest

SDD: System Design Document

Segmentation: The process of identifying and outlining suspicious regions at pixel level

U-Net: A deep learning architecture commonly used for biomedical image segmentation

Inference: The process of running a trained model on new input data to obtain predictions

MTL: A machine learning approach in which a single model is trained to perform multiple related tasks simultaneously using shared feature representations.

2. Current Software Architecture

This project does not directly replace one specific existing software system. Instead, it addresses a problem that is currently handled in a mostly manual or semi-manual way. In the current situation, breast images are reviewed by medical professionals using their own expertise and imaging tools, and suspicious regions are interpreted through visual examination. AI support, when available, is often external to the local workflow or limited to isolated research models rather than a single integrated desktop prototype.

Because of this, the current solution can be considered ad-hoc from the perspective of this project. The steps of image preparation, suspicious region identification, benign/malignant assessment, and visual explanation are not brought together in one unified student-built application. The absence of such an integrated flow is one of the main reasons for proposing this system. In the project proposal, the need for more consistent and efficient evaluation, reduced human error, and better support for diagnostic decision making was clearly stated.

3. Proposed Software Architecture

The proposed system follows a modular layered architecture. At a high level, it consists of a presentation layer, an application control layer, and several specialized backend subsystems responsible for preprocessing, model inference, explainability, and result management. This structure was selected because it provides clear subsystem boundaries and allows different parts of the system to be developed in parallel.

The user interacts with the system through a desktop interface. Once an image is uploaded, the application controller starts the analysis workflow. The input first passes through the preprocessing subsystem, where the image is converted into the format expected by the trained model. After that, the processed image is sent to the inference subsystem. The inference subsystem runs the segmentation model to identify suspicious regions and then uses the extracted output for classification. Finally, the explainability subsystem generates a visual explanation, and all outputs are returned to the interface for display.

In short, the general flow is:

User Interface → Preprocessing → Segmentation Model → Classification Model → Explainability Module → Result Display

3.1 Overview

The proposed system is organized as a modular layered desktop application for breast image analysis. At the highest level, the architecture is divided into four main concerns:

presentation, application control, analysis, and result management. This separation keeps the user-facing functionality independent from the internal AI pipeline and makes the system easier to understand, maintain, and extend. The Presentation Layer contains the Desktop UI, which is responsible for image selection, pipeline initiation, result visualization, and export requests. The Application Control Layer contains the Pipeline Controller, which coordinates the execution order of the analysis stages and handles error propagation between subsystems. The Analysis Layer contains the Preprocessing, MTL Inference, Explainability, and Analysis Result data. These packages prepare the input image, run segmentation and classification, generate the explainability visualization, and aggregate the outputs into a single result object. The Result Management Layer contains the Report Export package, which serializes the final outputs into a user-visible report file.

The system also interacts with a small set of local persistent artifacts. Input mammography images are read from the local file system, trained model weights are loaded locally at runtime, and exported reports are written back to local storage. Since the system is designed as a standalone desktop prototype, these artifacts remain on the same workstation and no remote service or central database is required during normal operation. This high-level organization is consistent with the detailed subsystem decomposition presented in the next subsection.

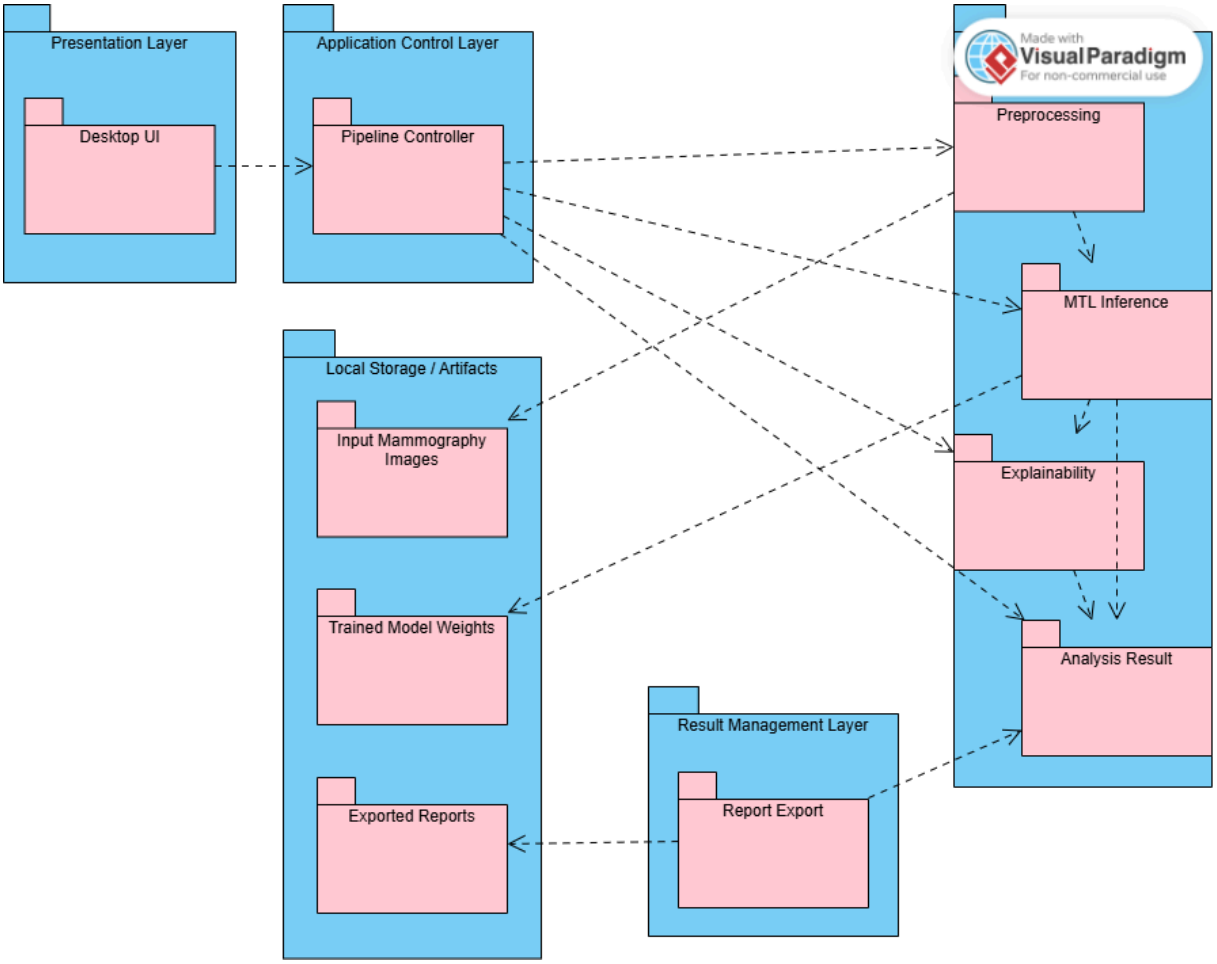

Figure 1. Package Diagram

3.2 System Decomposition

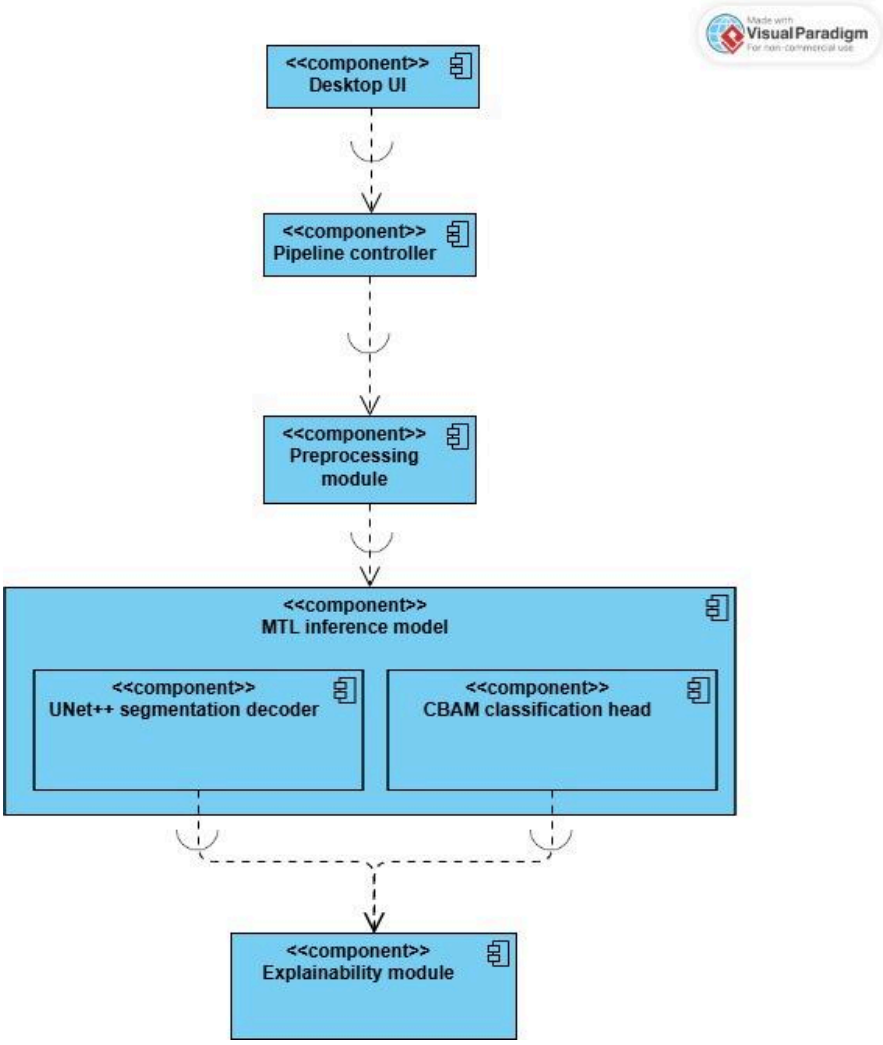

Figure 2. Component Diagram

The proposed system is decomposed into seven subsystems. Each subsystem has a clearly defined responsibility and communicates with adjacent subsystems through well-defined inputs and outputs.

Desktop UI The Desktop UI is the entry point of the system. It provides the graphical interface through which the medical professional uploads a mammography image, triggers the analysis pipeline, and views the results. It does not perform any computation, it delegates all processing to the pipeline controller and waits for results to display. It also exposes the export functionality for the user to save a report.

Pipeline Controller The Pipeline Controller is the central coordinator of the analysis workflow. It is triggered by the UI when the user clicks "Run Analysis" and is responsible for sequencing the pipeline stages in order. It calls each subsystem in sequence, passes outputs between stages, handles failures at any stage, and returns the final aggregated result to the UI. It has no AI logic of its own.

Preprocessing Module The Preprocessing Module receives a raw mammography image file path and produces a normalized tensor ready for model inference. Internally it applies morphological opening for artifact removal, Otsu thresholding and region-growing for pectoral muscle suppression, CLAHE for contrast enhancement, zero-padding to square dimensions, resizing to 224×224, and Z-score normalization. It is implemented using MONAI transforms, which ensure that any geometric operation applied to the image is simultaneously applied to any associated mask, preventing spatial misalignment.

MTL Inference Model The MTL Inference Model is the core AI component. It receives the preprocessed tensor and produces two outputs simultaneously in a single forward pass: a binary segmentation mask delineating the suspicious region, and a classification result indicating benign or malignant with an associated confidence score. Internally it consists of a shared EfficientNet-B0 encoder whose learned feature maps are routed to two branches, a UNet++ decoder branch for segmentation and a CBAM-enhanced classification head for diagnosis. The joint loss during training is a weighted combination of Dice Loss and Binary Cross-Entropy.

Explainability Module The Explainability Module receives the preprocessed image tensor and the trained MTL model and produces a Grad-CAM++ heatmap overlaid on the original image. It operates post-inference by hooking into the final convolutional layer of the shared EfficientNet-B0 encoder, computing the gradient of the classification score with respect to the feature maps, and generating a spatial activation map that highlights which regions most influenced the model's decision. This step is triggered after the MTL model completes its forward pass and adds approximately 0.29 seconds of overhead.

Export Report The Export Report subsystem receives the Analysis Result data and serializes it into a downloadable report file on the user's local file system. The report includes the annotated image, segmentation mask overlay, classification label, confidence score, and explainability heatmap.

3.3 Hardware/Software Mapping

The proposed system is deployed as a standalone desktop application on a single user workstation. All major software elements of the system run on the same machine, which serves both as the interaction point for the user and as the computation environment for the analysis pipeline.

The Desktop UI, Pipeline Controller, Preprocessing Module, MTL Inference Module, Explainability Module, and Report Export Module are all executed locally on the user’s computer. The user interacts with the application through the Desktop UI, which runs within the local operating system and Python runtime environment. When the user starts an analysis, the Pipeline Controller coordinates the execution of the internal modules in sequence and returns the final result to the interface for display.

Since the entire system is deployed on a single node, there are no network-based communication issues during normal operation. The design does not depend on remote

servers, cloud services, or external databases at runtime. As a result, problems such as inter-node synchronization, network latency, and distributed fault handling are avoided.

From the hardware perspective, the main requirement is a workstation with enough CPU power, memory, and storage to run the application and load the trained model. GPU support is optional. If a compatible GPU is available, it can be used to accelerate inference and reduce response time. Otherwise, the system can still operate entirely on CPU, although execution may take longer. In both cases, the logical mapping of the software components remains the same because the GPU acts only as a local acceleration resource rather than a separate deployment node.

Software reuse introduces some integration concerns. Because the system depends on external libraries such as PyTorch, MONAI, and pytorch-grad-cam, the compatibility of package versions must be managed carefully. Model loading, preprocessing behavior, and explainability generation all depend on a consistent local software environment. For this reason, the project should be distributed with fixed dependency versions and a clearly defined Python environment configuration. This helps ensure that the system behaves consistently across different machines.

In summary, the hardware/software mapping of the system is intentionally simple: one workstation, one local application, one local model artifact, and local storage for both inputs and exported outputs. This design supports the project's goals of usability, maintainability, portability, and straightforward deployment.

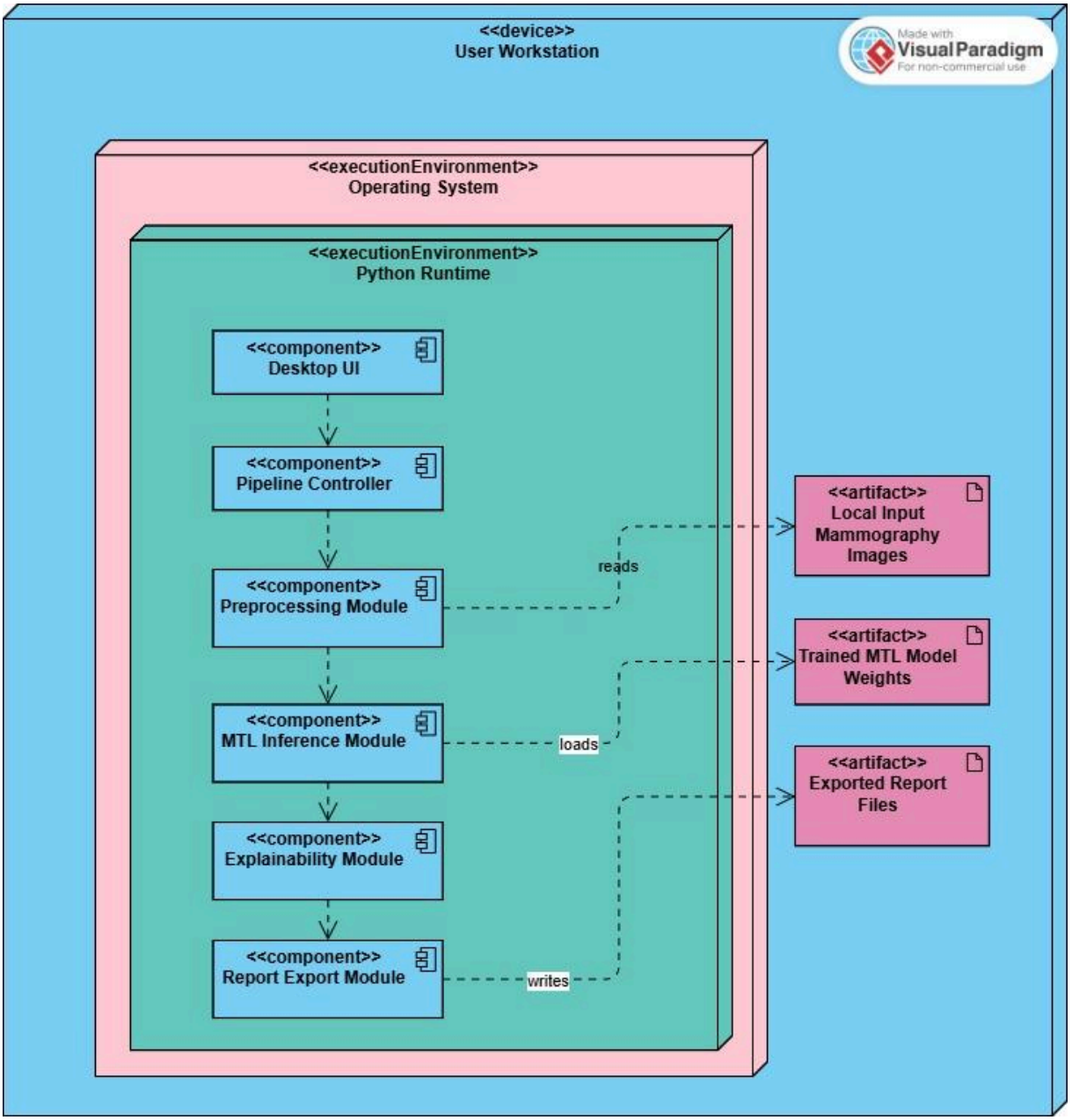

Figure 3. Deployment Diagram

3.4 Persistent Data Management

The proposed system has limited persistent data requirements because it is designed as a standalone desktop application running on a single workstation. Unlike a server-based information system, it does not require a central relational database or a distributed storage layer during normal operation. Instead, persistent data is managed primarily through the local file system. This design choice is consistent with the overall architecture of the system, where all major components execute locally and generated reports are saved on the user's machine.

The main persistent data items in the system are the input mammography image selected by the user, the trained MTL model weight file, and the exported report files generated after analysis. The input image is not created by the system, but it is read from a persistent location on the local file system. The trained model is also stored persistently as a local artifact. The report export subsystem creates persistent output files that contain the original image,

segmentation overlay, classification label, confidence score, and Grad-CAM++ visualization. These files are written to a user-selected output directory on the same workstation.

No relational or NoSQL database is required for the operational version of the system. This is because the application does not manage multiple users, long-term case histories, shared records, or remote queries. The core analysis result is represented internally , which aggregates the segmentation mask, classification label, confidence score, explainability heatmap, and original image. However, this object exists only in memory during execution and is not stored persistently unless the user explicitly exports a report.

From a data structure perspective, the persistent storage scheme is file-based rather than table-based. The system uses the following logical persistent data structures:

- **Input image file:** local mammography image
- **Model artifact:** trained MTL model weights stored as a PyTorch model file
- **Exported report file:** generated report containing visual and textual analysis results

The encapsulation of persistent data access is handled by the relevant subsystems rather than exposed directly to the UI. The Preprocessing Module is responsible for reading the selected input image, the MTL Inference Module is responsible for loading the trained model artifact, and the Report Export Module is responsible for writing the final report to persistent storage. This keeps file handling localized within the appropriate subsystem and improves maintainability by preventing uncontrolled direct access to stored data from multiple parts of the application.

Because the system operates on medical imaging data, persistent storage should be kept minimal and controlled. The design goal is to avoid storing unnecessary patient-related information. Only the files explicitly selected by the user and the outputs intentionally exported by the user are kept persistently. Intermediate tensors, masks, and temporary results are processed in memory and discarded after the analysis session ends unless they are included in the exported report.

3.5 Access Control and Security

The proposed system follows a simple and localized access control model because it is designed as a standalone desktop application running on a single workstation. In contrast to a multi-user hospital information system or a web-based medical platform, the system does not provide shared remote access, concurrent sessions, or centrally managed user roles. Its main actor is the authorized local user who runs the application on the workstation and interacts with the software through the desktop interface. This approach is consistent with the overall system architecture, in which all major components execute locally and persistent data is stored on the same machine.

Since the application is not deployed as a client-server or cloud-based system, it does not include an internal authentication mechanism such as application-level usernames, passwords, or token-based login. Instead, authentication is delegated to the operating system environment. In practical terms, access to the software and its files is controlled by the workstation's own login protection and file-system permissions. Anyone who is authorized to use the machine and has access to the application directory and relevant files is considered an authorized user of the system. This design is appropriate for the prototype scope of the

project, especially because the system is intended as a desktop-based application rather than a networked service.

From an operational perspective, the user is allowed to select an input image, start the analysis pipeline, inspect the generated outputs, and export the final report. The software itself performs file reading and writing only where necessary for execution. More specifically, the preprocessing subsystem reads the selected image file, the inference subsystem loads the trained model artifact, and the report export subsystem writes the generated output to a user-selected location. Intermediate outputs such as tensors, temporary masks, and internal result objects are not intended for direct manual access and remain under the control of the internal software modules during runtime. This helps keep access to data structured and limited to the parts of the system that actually require it.

Security considerations in this project focus primarily on confidentiality, controlled local access, and minimal retention of sensitive data. Because the application operates on medical imaging data, the design avoids unnecessary persistent storage. Only the files explicitly selected by the user and the files intentionally exported by the user are stored persistently. Intermediate analysis data is kept in memory during execution and discarded when the session ends unless it becomes part of an exported report. This reduces the risk of unintended long-term exposure of medical analysis data on the local machine.

The current system does not transmit data over a network during normal operation. As a result, many common security concerns associated with distributed systems, such as session hijacking, man-in-the-middle attacks, unauthorized API access, and remote service abuse, are outside the operational scope of this version. Security therefore depends mainly on the trustworthiness of the local workstation environment, including operating system security, device access control, disk protection, and proper handling of exported files by the user. This is consistent with the design decision to keep the system fully local and independent from remote servers or external databases at runtime.

Application-level encryption is not included in the current prototype. Data in transit encryption is not required because the system does not exchange information over a network during runtime. Data at rest is protected only to the extent provided by the underlying operating system and storage environment. If stronger protection is needed in a future version, exported reports and local model/data files could be protected through operating-system-level disk encryption, secure folders, or password-protected storage mechanisms. Similarly, explicit cryptographic key management is not part of the present design because the system does not manage encrypted communications, remote credentials, or internal secret-sharing mechanisms.

Because the project is situated in a medical imaging context, privacy-aware usage is important even in a prototype environment. During development and evaluation, the system should be used with public, anonymized, or de-identified datasets. Exported reports should contain only the information required for analysis and interpretation, such as the original image view, segmentation overlay, classification result, confidence score, and explainability visualization. Patient-identifiable metadata should not be preserved or displayed unless it is strictly required and explicitly handled under an approved data governance policy. This is aligned with the project's broader goal of avoiding unnecessary patient-related information in persistent storage while still providing interpretable diagnostic support output.

In summary, the access control and security strategy of the proposed system is intentionally lightweight, local, and appropriate for a standalone prototype. The design relies on workstation-level authentication, file-system permissions, minimal persistent storage, and the absence of unnecessary network exposure. This keeps the security model consistent with the project's desktop architecture while still addressing the main confidentiality and privacy concerns relevant to medical image analysis.

3.6 Global Software Control

The system uses a linear, event-driven control flow. The entire pipeline is initiated by a single user action clicking the "Run Analysis" button in the Desktop UI. There is no background processing, no polling, and no concurrent execution between pipeline stages.

When the user clicks "Run Analysis", the Desktop UI calls PipelineController.run (image_path), The Pipeline Controller then executes each stage sequentially in the following order:

PreprocessingModule.process(image_path): returns a normalized tensor

MTLModel.forward(tensor): returns segmentation mask + classification label + confidence score simultaneously in one forward pass.

ExplainabilityModule.generate(tensor, model): returns Grad-CAM++ heatmap

UI.display(result): renders all outputs to the results screen.

The pipeline is strictly sequential; no stage begins until the previous one completes. No parallelism is introduced between stages because the output of each stage is the required input of the next.

Failure handling: If any stage raises an exception, the Pipeline Controller catches it, halts the pipeline immediately, and surfaces a user-readable error message to the UI. The pipeline does not attempt partial recovery; a failure at any stage produces no output, and the user is informed to retry.

Synchronization: Since the application runs on a single machine with no network dependency, there are no distributed synchronization concerns. The only concurrency consideration is that model inference (MTLModel.forward) is a blocking call that may take several seconds on CPU. To prevent the UI from freezing during inference, the pipeline is executed in a background thread while the UI displays a loading indicator.

3.7 Boundary Conditions

Start-up:

When the application launches, the Pipeline Controller performs an initialization check before the UI becomes interactive. It verifies that the trained MTL model weights file exists at the expected local path, If the file is found, the weights are loaded into memory using torch.load() and the application proceeds to display the main screen in a ready state. If the weights file is missing or corrupted, the application displays a clear error message informing the user that the model file could not be found, and the "Run Analysis" button remains disabled. The application does not crash, it opens in a degraded state where image loading is possible but analysis cannot be triggered.

Shutdown

When the user closes the application, any in-memory data including the loaded tensor, segmentation mask, classification result, heatmap, and Analysis Result data is discarded. No session data is written to disk on shutdown. If the user closes the application while a pipeline run is in progress, the background inference thread is terminated and no partial results are saved. The application exits cleanly without requiring a save prompt.

Error Behavior

The following error conditions are handled explicitly:

Unsupported file format: If the user attempts to load a file in an unsupported format, the system displays an error message and returns to the ready state without proceeding to analysis. The "Run Analysis" button remains disabled until a valid file is loaded.

Preprocessing failure: If the Preprocessing Module fails (for example due to a corrupted image or an unexpected pixel format), the Pipeline Controller catches the exception, halts the pipeline, and displays a user-readable message indicating that the image could not be processed. The application returns to the ready state.

Inference failure: If the MTL model raises an exception during the forward pass (for example due to insufficient memory), the Pipeline Controller catches it, halts the pipeline, and informs the user that analysis could not be completed. The application does not crash.

Explainability failure: If Grad-CAM++ generation fails after a successful inference, the system displays the classification result and segmentation mask as normal but notifies the user that the explainability visualization could not be generated. This is consistent with UC5 Scenario 2 defined in the RAD.

Export failure: If the user attempts to export results to a path they do not have write permission for, the Export Report module catches the filesystem exception and informs the user without affecting the displayed results.

4. Subsystem Services:

PreprocessingModule:

Exposes: process(image_path: str) -> tensor: ndarray

Receives a file path to a mammography image in PNG, TIFF, or DICOM-converted format. Returns a normalized float32 numpy array of shape (1, 224, 224) representing a single-channel grayscale image. All geometric transforms (padding, resizing) are applied jointly to the image using MONAI transforms to preserve spatial integrity.

MTLModel:

Exposes: forward(tensor: ndarray) ->(mask: ndarray, label: str, score: float)

Receives the preprocessed tensor of shape (1, 224, 224). Executes a single forward pass through the shared EfficientNet-B0 encoder and returns three values simultaneously:

- mask: binary segmentation mask of shape (224, 224) as uint8, where 1 = suspicious region and 0 = background.
- label: string, either "Benign" or "Malignant".
- score: float between 0.0 and 1.0 representing the model's confidence in the malignant prediction.

ExplainabilityModule:

Exposes: generate(tensor: ndarray, model: MTLModel, original_image: ndarray) -> heatmap: ndarray

Receives the preprocessed tensor, the trained MTLModel instance, and the original image. Hooks into the final convolutional layer of the shared encoder via jacobgil/pytorch-grad-cam, computes Grad-CAM++ gradients with respect to the classification output, and returns a heatmap of shape (224, 224, 3) as uint8 — a color-coded overlay superimposed on the original image for display.

ExportReport:

Exposes: create_report(pipeline_outputs, output_path: str) -> None

Receives the aggregated pipeline outputs and a user-selected output path. Compiles the original image, segmentation mask overlay, classification label, confidence score, and explainability heatmap into a report file and saves it to the local filesystem.

Desktop UI:

Exposes: display(pipeline_outputs) ->None

Receives the pipeline outputs from the Pipeline Controller and renders the original image, segmentation overlay, classification label with confidence score, and explainability heatmap in the results screen.

5. References

1. What is Package Diagram, https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-package-diagram/ Accessed on 22 March 2025.
2. What is Component Diagram, https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/ Accessed on 22 March 2025.
3. What is Deployment Diagram, https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-deployment-diagram/ Accessed on 22 March 2025.

4. Lee, R. S., Gimenez, F., Hoogi, A., Miyake, K. K., Gorovoy, M., & Rubin, D. L. (2017). A curated mammography data set for use in computer-aided detection and diagnosis research. Scientific Data, 4, 170177. https://doi.org/10.1038/sdata.2017.177 (TCIA collection DOI: 10.7937/K9/TCIA.2016.7O02S9CY)

5. Zhou, Z., Siddiquee, M. M. R., Tajbakhsh, N., & Liang, J. (2018). UNet++: A Nested U-Net Architecture for Medical Image Segmentation. Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support, Springer, Cham. https://arxiv.org/abs/1807.10165

6. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. Proceedings of the 36th International Conference on Machine Learning (ICML), Vol. 97, pp. 6105-6114. https://arxiv.org/abs/1905.11946

7. Woo, S., Park, J., Lee, J. Y., & Kweon, I. S. (2018). CBAM: Convolutional Block Attention Module. Proceedings of the European Conference on Computer Vision (ECCV), pp. 3-19. https://arxiv.org/abs/1807.06521

8. Gildenblat, J. and contributors. (2021). PyTorch library for CAM methods. GitHub repository. https://github.com/jacobgil/pytorch-grad-cam

9. MONAI Consortium. (2020). MONAI: Medical Open Network for AI. GitHub repository. https://github.com/Project-MONAI/MONAI

10. ASG-MammoNet, EfficientNet-B0 + CBAM classification on CBIS-DDSM (99.78% accuracy, AUC 0.996). Frontiers in Signal Processing, 2025. https://www.frontiersin.org/journals/signal-processing/articles/10.3389/frsip.2025.1672569/full

11. CLAHE preprocessing + EfficientNet + CBAM + Grad-CAM++ pipeline for mammography. MDPI Diagnostics, 2026. https://www.mdpi.com/2075-4418/16/1/105

12. Multi-task learning for breast tumor segmentation and classification, MTL synergy results (+4.13% segmentation recall, +2.34% classification accuracy). PMC. https://pmc.ncbi.nlm.nih.gov/articles/PMC8902030/

13. Multi-task learning framework for automated segmentation and classification of breast tumors from ultrasound images. ResearchGate. https://www.researchgate.net/publication/358439067_A_Multi-Task_Learning_Framework_for_Automated_Segmentation_and_Classification_of_Breast_Tumors_From_Ultrasound_Images

14. Grad-CAM++ explainability validation metrics (MGT score, 0.29s generation time). PMC. https://pmc.ncbi.nlm.nih.gov/articles/PMC12785440/

15. XAI validation metrics for mammography AI, Matching Ground Truth (MGT) score, top models achieving 0.86. arXiv. https://arxiv.org/html/2512.00129v1